

Подсистема работы с проектами приложений на платформе (sbis-loader)

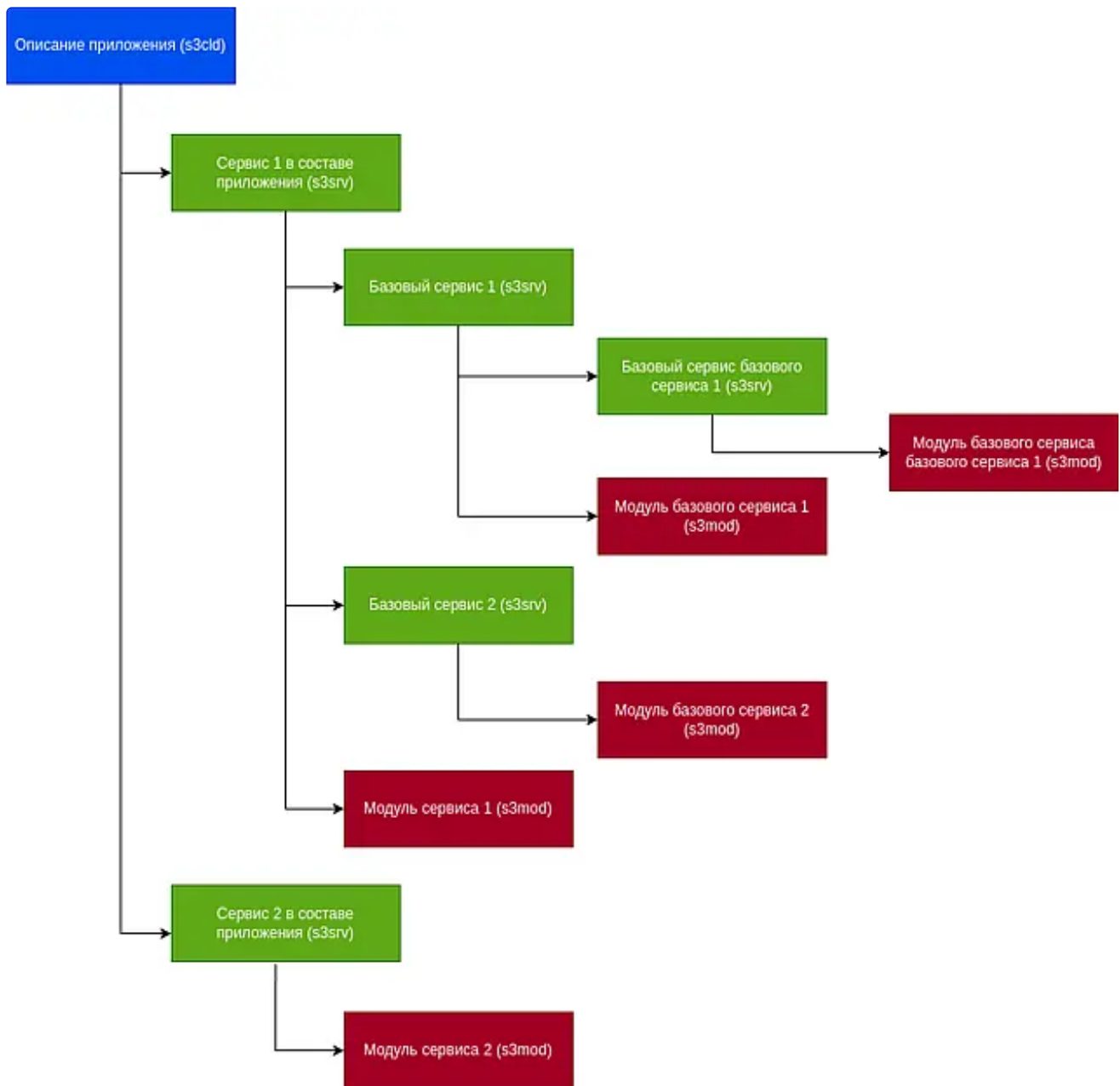
На данный момент структура проектов на базе платформы описывается в файлах трех форматов:

1. Файл описания приложения в целом - расширение s3cld
2. Файл описания сервиса в составе приложения - расширение s3srv
3. Файл описания модуля БЛ или UI - расширение s3mod

Файлы описания приложений могут иметь ссылки друг на друга и в конечном итоге образуют древовидную структуру, корнем которой является s3cld-файл. Возможны следующие виды ссылок:

1. Из файла s3cld на файлы s3srv
2. Из файла s3srv на другие файлы s3srv (наследование сервисов) и файлы s3mod

При этом один и тот же сервис и один и тот же модуль могут появляться в проекте неоднократно. Например, сервис может быть родительским для нескольких других сервисов, а модуль может быть подключен в неограниченное число сервисов.



Каждый элемент дерева проекта имеет уникальный идентификатор (uuid). При этом на файлы налагается жесткое требование уникальности идентификатора для каждого узла дерева проекта и уникальности его расположения в файловой системе.

Связи между элементами дерева проекта могут быть разных типов. Выделяются:

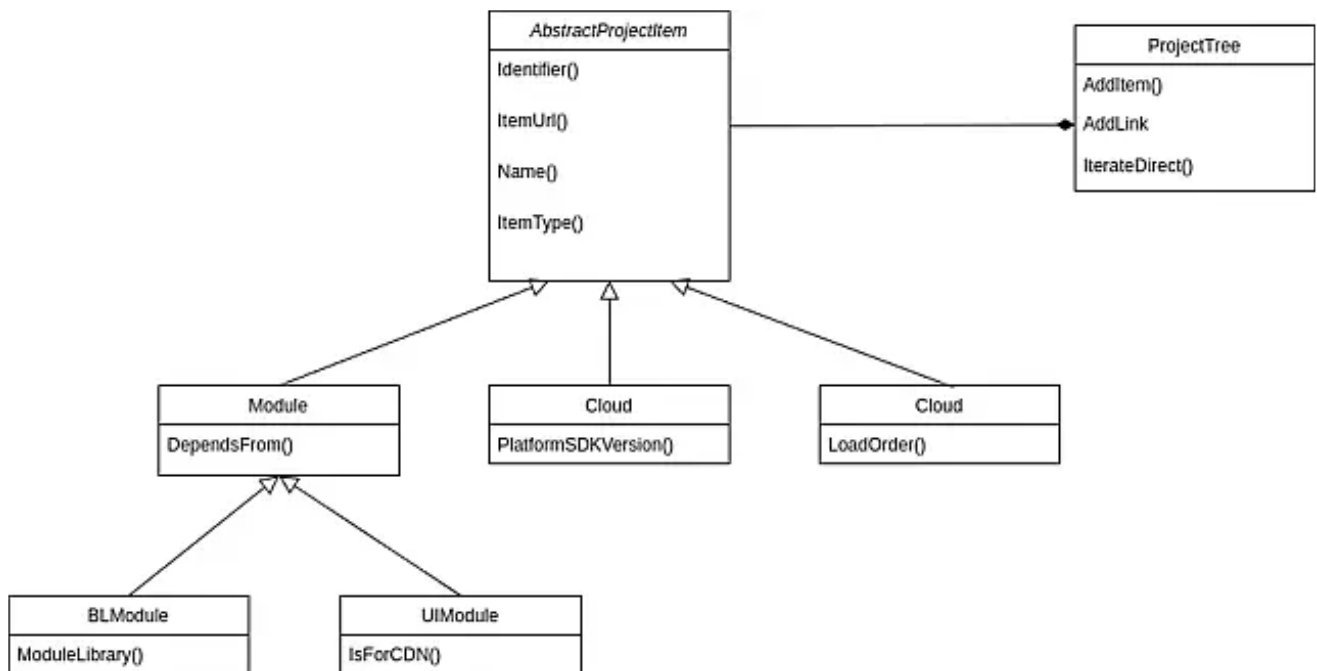
1. "Прямые связи" - когда дочерний элемент напрямую подключен в родительскому с указанием пути до файла с его описанием относительно файла с описанием родительского элемента
2. "Сильные связи" - когда дочерний элемент напрямую подключен в родительскому, но без указания пути до файла с его описанием относительно файла с описанием родительского элемента
3. "Слабые связи" - когда дочерний элемент напрямую НЕ подключен в родительскому, но в процессе загрузки находится по зависимостям других элементов и включается в загруженный проект.

Связи двух последних типов "разрешаются" в файловые пути при помощи артефактов, производимых [Module Manager](#)-ом.

Для программной работы с вышеуказанной структурой разработан набор классов в библиотеке sbis-loader.

Выделяются две основные абстракции:

1. Проект в целом (класс `Id::ProjectTree`)
2. Узел дерева проекта (абстрактный класс `Id::AbstractProjectItem`) и конкретные типы узлов (классы-наследники `Id::AbstractProjectItem`)



Объект класса ProjectTree является единственным владельцем некоторого множества уникальных объектов классов-наследников AbstractProjectItem, а также набора связей между ними (структура `Id::TreeLink`). Каждый объект класса-наследника AbstractProjectItem, если он включен в какой-либо объект ProjectTree, имеет ссылку на объект класса-владельца.

Класс ProjectTree имеет API для выполнения обходов дерева проекта различными способами при помощи вызываемых (callable) объектов (обход потомков данного узла - метод `IterateChildrenOf`, обход в глубину от данного узла - метод `IterateDirect`, обход в глубину от данного узла с отбором элементов определенного типа - метод `IterateDirectFiltered`) и поиска элементов, удовлетворяющих переданному пользователем критерию (метод `FindIf`). При выполнении обхода в глубину надо быть готовым к тому, что одни и те же узлы будут встречаться многократно.

```

1  Id::ProjectTree tree;
2  tree.IterateDirectFiltered< Id::BLModule >( tree.Root(), [](
3      Id::BLModule const& mod )
4  {
5      // обходим все модули бизнес-логики в составе проекта и печатаем их
    название
6      // вероятно, что многие модули будут посещены многократно
7      std::wcout << mod.Name() << std::endl;
8  } );
  
```

Также класс ProjectTree имеет методы для добавления и удаления связей между элементами дерева проекта. Не гарантируется, что в любой момент времени все узлы дерева достижимы из его корня, возвращаемого методом `Root()` (в корне дерева проекта всегда находится объект с типом `Id::Cloud`). Соответственно, при выполнении обхода в глубину такие узлы не будут посещены обходчиком. Для удаления недостижимых узлов после каких-либо манипуляций с деревом проекта следует применять методов `RemoveUnreachable` или `Normalize`.

Загрузка проектов (построение объекта ProjectTree) осуществляется при помощи классов-наследников класса **Id::AbstractProjectLoader**. API данного класса позволяет загружать или проект в целом, или только один сервис (файл `s3srv`). В последнем случае, чтобы дерево проекта было полным, формируется вспомогательный виртуальный корневой узел с типом `Id::Cloud`, для которого добавляется связь на загруженный сервис. На момент написания данного текста в библиотеке присутствует два основных загрузчика:

1. Класс `Id::FilesystemProjectLoader`, выполняющий загрузку проекта из файловой системы
2. Класс `Id::FlatProjectLoader`, выполняющий загрузку из вкомпилированных метаданных (используется в мобильных и десктопных решениях для ускорения старта приложения)

При наличии в составе проекта "сильных" и "слабых" связей (см. выше) для успешной работы загрузчика требуется передача в его методы объекта класса-наследника **Id::AbstractItemSpecResolver**, ответственного за поиск путей к файлам с описанием узла дерева по его типу, имени и идентификатору.

Ко всем типам узлов дерева, за исключением корневого (`Id::Cloud`) можно прикреплять список дополнительных ресурсов, необходимых для успешной работы данного компонента. Такими ресурсами могут быть динамические библиотеки, `pip`-пакеты, файлы и каталоги. Для каждого ресурса можно указать точку развертывания в каталоге приложения. Описывается каждый ресурс классом **Id::BinaryResource**.

Многие элементы дерева проекта, либо отношения между ними, являются платформозависимыми, т.е. этот элемент или это отношение имеет место только в том случае, если целевая платформа удовлетворяет определенным

критериям. Такого рода критерии описываются структурой **Id::PlatformConfig**. Она представляет собой набор битовых масок, соответствующих тому или иному аспекту целевой платформы. На момент написания данной статьи в ее составе есть маски для архитектуры процессора, операционной системы, компилятора, типа линковки, типа сборки (релиз/отладка). Платформозависимыми могут быть:

1. Сервисы
2. Модули
3. Межмодульные зависимости
4. Ресурсы элементов дерева проекта

Т.е., например, можно указать, что зависимость модуля А от модуля Б есть только на 32-х битном Windows, а не везде.